

Tema 1. Exigencias Computacionales del Procesamiento Digital de la Información

Álgebra de Boole: fundamentos matemáticos, leyes, representaciones canónicas, simplificación y puertas lógicas.

FUNDAMENTOS DIGITALES

Definición del Álgebra de Boole

El álgebra de Boole es **una estructura matemática** construida a partir de un conjunto de **elementos** sobre los que se definen **operadores**, estableciendo **postulados o axiomas** que los relacionan. Se parte de una estructura algebraica **(B, +, *)** con dos operaciones: suma (+) y producto (*).

Postulado I — Cierre

$\forall a, b \in B: a + b \in B$ y $a * b \in B$. El conjunto B es cerrado respecto a ambas operaciones.

Postulado II — Elemento Identidad

$\forall a \in B: a + 0 = a$ y $a * 1 = a$. El 0 es identidad en + y el 1 en *.

Postulado III — Conmutatividad

$\forall a, b \in B: a + b = b + a$ y $a * b = b * a$. Ambas operaciones son conmutativas.

Postulados de Huntington (II)

Postulado IV — Distributividad

Cada operación es distributiva respecto a la otra:

$$a * (b + c) = (a * b) + (a * c)$$

$$a + (b * c) = (a + b) * (a + c)$$

Postulado V — Complemento

$\forall a \in B$ existe un elemento complementario a' :

$$a * a' = 0$$

$$a + a' = 1$$

Postulado VI — Número de Elementos

En B existen al menos dos elementos diferentes: **$a \neq b$** .

Leyes y Teoremas del Álgebra de Boole

Principio de Dualidad

Cada identidad permanece válida si se intercambian 0 ↔ 1 y + ↔ *.

Idempotencia

$a*a=a$, $a+a=a$, $a*0=0$, $a+1=1$. Definen la suma y producto lógico.

a	+	b	S
0	+	0	0
0	+	1	1
1	+	0	1
1	+	1	1

a	*	b	S
0	*	0	0
0	*	1	0
1	*	0	0
1	*	1	1

Absorción

$a+a*b=a \rightarrow a*(1+b)=a*1=a$

$a*(a+b)=a \rightarrow a*a+a*b=a$

$a+ab=a(1+b)=a*1=a$

Asociatividad

$a+(b+c)=(a+b)+c$ y $a*(b*c)=(a*b)*c$.

Involución

$a''=a$. El doble complemento devuelve el elemento original.

Teoremas Fundamentales

Teorema de Adyacencia

$$a * b + a * b' = a$$

$$(a + b) * (a + b') = a$$

Teorema de Expansión de Shannon

Toda función se puede descomponer:

$$f(a, b, c, \dots) = a * f(1, b, c, \dots) + a' * f(0, b, c, \dots)$$

Siendo $f(1, b, c, \dots)$ la función resultante de sustituir, en la función original, todas las a por 1, y las a' por 0.

El segundo término, $f(0, b, c, \dots)$ es la función resultante de sustituir las a' por 1 y las a por 0.

Teorema De Morgan (Augustus De Morgan)

Permite convertir sumas en productos y viceversa mediante complementación:

$$(a + b)' = a' * b'$$

$$(a * b)' = a' + b'$$

Es fundamental para la conversión entre minitérminos y maxitérminos, y para implementar funciones con puertas NAND/NOR.

Álgebra de Boole Bivalente

Una **variable** es un símbolo que representa cualquier elemento del conjunto $B = \{0,1\}$. Una **función lógica** relaciona variables mediante suma lógica, producto lógico y negación: $f = f(a, b, c, \dots)$.

Ejemplo

Sea $f(c,b,a) = ba + cba' + b'a + a$. Sustituyendo $f(1,1,1)$:

$$f = (1 * 1) + (1 * 1 * 0) + (0 * 1) + (1) = 1$$

Tabla de Verdad de $f(c,b,a)$

c	b	a	$f(c, b, a)$
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

Funciones Equivalentes y Tablas de Verdad

Dos funciones lógicas son **equivalentes** si tienen la **misma tabla de verdad**. Las tablas de verdad representan el valor de la función para cada combinación posible de variables.

Ejemplo: ¿Son equivalentes f_1 y f_2 ?

$f_1 = ba + cba' + b'a + a$

$f_2 = cb + a$

c	b	a	f_1	f_2
0	0	0	0	0
0	0	1	1	1
0	1	0	0	0
0	1	1	1	1
1	0	0	0	0
1	0	1	1	1
1	1	0	1	1
1	1	1	1	1

Verificación De Morgan

Demostración de $(a+b)' = a'*b'$:

a	b	$(a+b)'$	$a'*b'$
0	0	1	1
0	1	0	0
1	0	0	0
1	1	0	0

☐ Ambas funciones son equivalentes: misma tabla de verdad.

Forma Canónica: Minitérminos y Maxitérminos

Un **término canónico** es aquel en el que aparecen **todas las variables** de la función, en forma directa o negada. Una función formada únicamente por términos canónicos se llama **función canónica**.

Sea la función: $f(a, b, c) = abc' + ab'c' + ac + b'c$

Términos canónicos Términos NO canónicos

Minitérminos (productos canónicos)

Términos producto donde aparecen todas las variables. La función canónica se expresa como **suma de minitérminos**: $f = \sum m_i$. Se coloca un 1 en cada minitérmino donde $f=1$.

Maxitérminos (sumas canónicas)

Términos suma donde aparecen todas las variables. La función canónica se expresa como **producto de maxitérminos**: $f = \prod M_i$. Se coloca un 0 en cada maxitérmino donde $f=0$.

Para n variables existen 2^n minitérminos y 2^n maxitérminos posibles. El índice del maxitérmino M_i es el complemento del número binario que representa al minitérmino m_i .

Ejemplo: Obtención de la Forma Canónica

Dada la tabla de verdad siguiente, se obtiene la función canónica usando minitérminos y maxitérminos:

Para los minitérminos se toman aquellos en que la función toma el valor 1 y para los Maxitérminos se toman aquellos en que los valores son 0.

c	b	a	f	m_i	M_i
0	0	0	0	m_0	M_7
0	0	1	1	m_1	M_6
0	1	0	0	m_2	M_5
0	1	1	1	m_3	M_4
1	0	0	1	m_4	M_3
1	0	1	0	m_5	M_2
1	1	0	1	m_6	M_1
1	1	1	1	m_7	M_0

Por Minitérminos (f=1)

$$f = \sum m_i = m_1 + m_3 + m_4 + m_6 + m_7$$

$$f = c'b'a + c'ba + cb'a' + cba' + cba$$

Por Maxitérminos (f=0)

$$f = \prod M_i = M_7 \cdot M_5 \cdot M_2$$

$$f = (c' + b' + a')(c' + b + a')(c + b' + a)$$

Conversión entre Minitérminos y Maxitérminos

Un maxitérmino es el complemento del minitérmino y viceversa. Aplicando las leyes de De Morgan se puede realizar la conversión:

$$\overline{m_i} = M_{2^n-1-i} \quad \overline{M_i} = m_{2^n-1-i}$$
$$\sum m_i = \overline{\prod M_i} \quad \prod M_i = \overline{\sum m_i}$$

- ❏ Para convertir de minitérminos a maxitérminos: tomar los minitérminos que **faltan** y expresarlos como producto de maxitérminos.

Ejemplo: Transformar $f = \Sigma_3(1,3,4,6,7)$ a maxitérminos

Los minitérminos que faltan son m_0 , m_2 y m_5 . Usando la relación $m_0=M_7$, $m_2=M_5$, $m_5=M_2$:

$$f = M_7 \cdot M_5 \cdot M_2$$

Funciones Incompletamente Definidas

Función Completamente Definida

Para cada combinación posible de variables, la función toma un valor determinado: 1 ó 0.

$$f_{\text{completa}}(c, b, a) = \sum_3(1, 3, 5, 6, 7)$$

Función Incompleta

Algunas combinaciones son **indiferentes** (puede ser 0 ó 1). Se representan con "**X**".

$$f_{\text{incompleta}}(c, b, a) = \sum_3(1, 3, 6, 7) + X(2, 4, 5)$$

Las condiciones indiferentes se aprovechan en la simplificación por Karnaugh para obtener grupos mayores.

$$f_{\text{completa}}(c, b, a) = \sum_3(1, 3, 5, 6, 7)$$

$$f_{\text{incompleta}}(c, b, a) = \sum_3(1, 3, 6, 7) + X(2, 4, 5)$$

c	b	a	Completa	Incompleta
0	0	0	0	0
0	0	1	1	1
0	1	0	0	X
0	1	1	1	1
1	0	0	0	X
1	0	1	1	X
1	1	0	1	1
1	1	1	1	1

Simplificación Algebraica de Funciones

Se simplifica aplicando las leyes del Álgebra de Boole. Ejemplo con $XZ'Y + (XZ'Y + ZX')(Y(Z+X) + Y'Z + Y'XZ')$:

Ejemplo:

Simplificar la siguiente función aplicando las leyes del Algebra de Boole.

$$XZ'Y + (XZ'Y + ZX')(Y(Z+X) + Y'Z + Y'XZ')$$

Comenzamos rompiendo paréntesis:

$$XZ'Y + (XZ'Y + ZX')(Y(Z+X) + Y'Z + Y'XZ')$$

$$XZ'Y + (XZ'Y + ZX')(YZ + YX + Y'Z + Y'XZ')$$

Eliminamos las variables en una suma que aparezcan en su forma normal y negada:

$$XZ'Y + (XZ'Y + ZX')(YZ + YX + Y'Z + Y'XZ')$$

$$XZ'Y + (XZ'Y + ZX')(Z(Y + Y') + YX + Y'XZ')$$

Desarrollamos el producto: $a*(b+c)=a*b+a*c$

$$XZ'Y + (XZ'Y + ZX')(Z + YX + Y'XZ')$$

$$XZ'Y + XZ'Y(Z + YX + Y'XZ') + ZX'(Z + YX + Y'XZ')$$

$$XZ'Y + XZ'YZ + XZ'YYX + XZ'YY'XZ' + ZX'Z + ZX'YX + ZX'Y'XZ'$$

Eliminamos los términos que tengan un producto entre una variable y su negada: $a*a'=0$:

$$XZ'Y + \cancel{XZ'YZ} + XZ'YYX + \cancel{XZ'YY'XZ'} + ZX'Z + \cancel{ZX'YX} + \cancel{ZX'Y'XZ'}$$

$$XZ'Y + XZ'YYX + ZX'Z$$

Eliminamos las variables repetidas en los términos producto: $a*a=a$

$$XZ'Y + XZ'YX + ZX'Z$$

$$XZ'Y + XZ'Y + ZX'$$

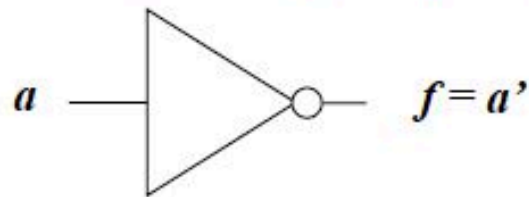
Quitando los términos repetidos: $XZ'Y + ZX'$

Puertas Lógicas Básicas

FUNCIONES BÁSICAS

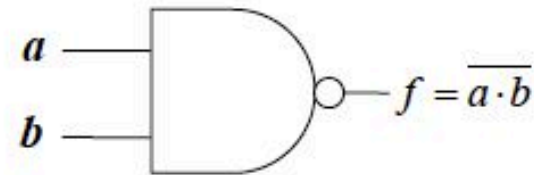
Funciones básicas=Puertas básicas

Inversor o Negador (NOT):



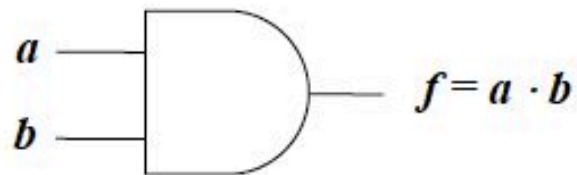
NOT		
a		f
0		1
1		0

Puerta NAND



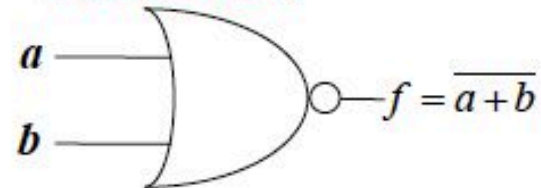
NAND		
a	b	f
0	0	1
0	1	1
1	0	1
1	1	0

Puerta AND



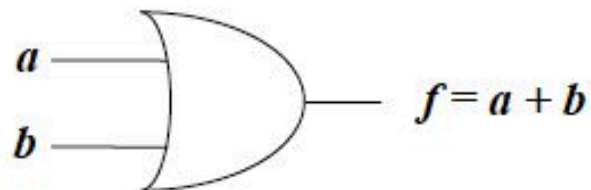
AND		
a	b	f
0	0	0
0	1	0
1	0	0
1	1	1

Puerta NOR



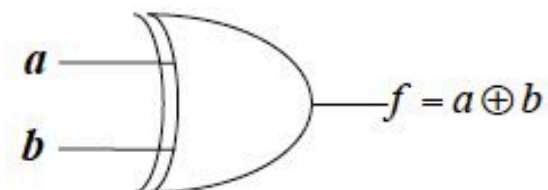
NOR		
a	b	f
0	0	1
0	1	0
1	0	0
1	1	0

Puerta OR



OR		
a	b	f
0	0	0
0	1	1
1	0	1
1	1	1

Puerta XOR (OR EXCLUSIVA)



XOR		
a	b	f
0	0	0
0	1	1
1	0	1
1	1	0

Representación Completa con Puertas NAND

Cualquier función lógica puede representarse mediante puertas NAND. El procedimiento es:

Ejemplo:

Representar la siguiente función utilizando únicamente puertas NAND

$$f_1 = XY\bar{Z} + XYZ + X(Y + \bar{Z})$$

Lo primero que debemos realizar es obtener una expresión mínima en forma de suma de productos.

$$XY\bar{Z} + XYZ + XY + X\bar{Z}$$

Lo siguiente es complementar dicha función 2 veces (al complementar 2 veces la función queda como está).

$$\overline{\overline{XY\bar{Z} + XYZ + XY + X\bar{Z}}}$$

Aplicar las leyes de De Morgan hasta que solo encontremos variables negadas o negaciones de productos.

$$f_2 = \overline{\overline{XY\bar{Z}} \cdot \overline{\overline{XYZ}} \cdot \overline{\overline{XY}} \cdot \overline{\overline{X\bar{Z}}}}$$

Comprobamos que ambas tablas de verdad coinciden:

T.5: Teoremas de DeMorgan:

$$\overline{X + Y} = \bar{X} \cdot \bar{Y}$$

$$\overline{X \cdot Y} = \bar{X} + \bar{Y}$$

X	Y	Z	XYZ'	XYZ	X(Y+Z')	f ₁	(XYZ')'	(XYZ)'	(XY)'	(XZ')'	f ₂
0	0	0	0	0	0	0	1	1	1	1	0
0	0	1	0	0	0	0	1	1	1	1	0
0	1	0	0	0	0	0	1	1	1	1	0
0	1	1	0	0	0	0	1	1	1	1	0
1	0	0	0	0	1	1	1	1	1	0	1
1	0	1	0	0	0	0	1	1	1	1	0
1	1	0	1	0	1	1	0	1	0	0	1
1	1	1	0	1	1	1	1	0	0	1	1

Mapas de Karnaugh (1954)

Método gráfico para simplificar funciones booleanas mediante tablas organizadas con el **código Gray** (código reflejado), donde celdas adyacentes difieren en una sola variable.

Karnaugh de 2 variables		
b \ a	0	1
0		
1		

Karnaugh de 3 variables				
c \ ab	00	01	11	10
0				
1				

Código **Gray** o reflejado.

Karnaugh de 4 variables				
cd \ ab	00	01	11	10
00				
01				
11				
10				

Código **Gray** o reflejado.

Karnaugh de 5 variables								
de \ abc	000	001	011	010	110	111	101	100
00								
01								
11								
10								

Código **Gray** o reflejado.

Simplificación con Karnaugh: Ejemplo de 3 Variables

a	b	c	$f(a, b, c)$
0	0	0	0
0	0	1	1
0	1	0	X
0	1	1	1
1	0	0	0
1	0	1	X
1	1	0	1
1	1	1	1

Karnaugh de 3 variables				
ab c	a' b ' 00	a' b 01	ab 11	ab' 10
c' 0	0	X	1	0
c 1	1	1	1	1

Se eliminan las variables que aparecen en su forma normal y negada simultáneamente.

La "c" aparece en ambas formas, por lo tanto quedará eliminada.
Lo mismo ocurre con la variable "a".
Este término queda como b .

Tanto la "a" como la "b" aparecen en ambas formas, por lo tanto quedarán eliminadas.
Este término queda como c .

La función simplificada queda como:

$$f = c + b$$

Simplificación con Karnaugh: Ejemplo de 4 Variables

Para la función de 4 variables con la tabla de verdad dada, el mapa de Karnaugh permite identificar tres grupos:

a	b	c	d	$f(a, b, c)$
0	0	0	0	1
0	0	0	1	0
0	0	1	0	1
0	0	1	1	1
0	1	0	0	1
0	1	0	1	0
0	1	1	0	1
0	1	1	1	1
1	0	0	0	1
1	0	0	1	0
1	0	1	0	1
1	0	1	1	0
1	1	0	0	0
1	1	0	1	0
1	1	1	0	0
1	1	1	1	0

Karnaugh de 4 variables				
ab \ cd				
	00	01	11	10
00	1	0	1	1
01	1	0	1	1
11	0	0	0	0
10	1	0	0	1

Las variables "c" y "a" aparecen en sus formas normal y negada.
Este término queda como $b'd'$.

La "d" aparece en ambas formas, al igual que la "b" por lo tanto estas dos variables quedarán eliminadas.
Este término queda como $a'c$.

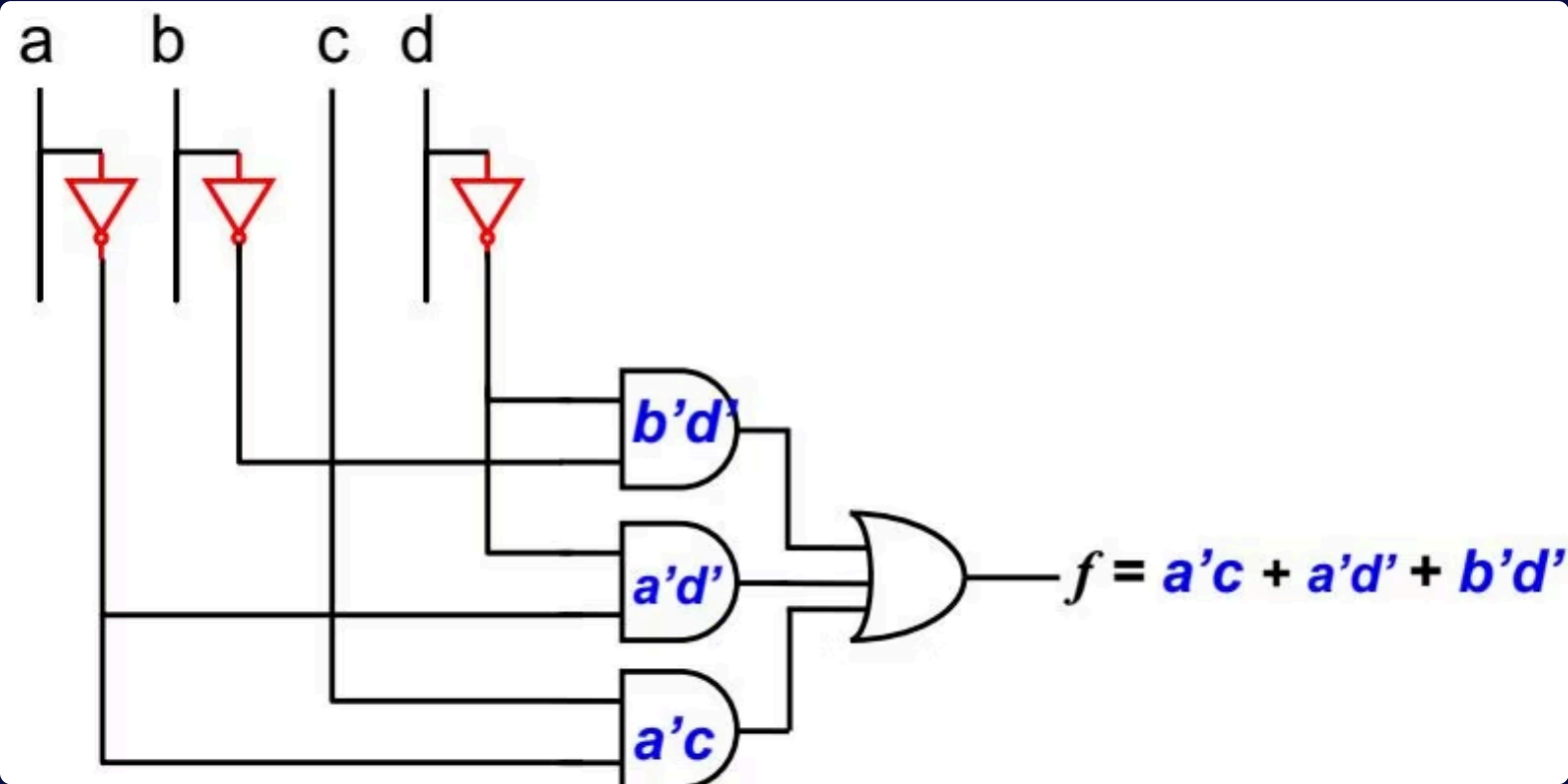
La variable "b" y la "c" aparecen en sus formas normales y negadas.
Este término queda como $a'd'$.

La función simplificada es:

$$f = a'c + a'd' + b'd'$$

Circuito Lógico Resultante

La función simplificada $f = a'c + a'd' + b'd'$ se implementa con inversores (NOT), puertas AND y una puerta OR:



Ejemplo con función incompleta

$f = \Sigma_3(0,6,7) + X(1,2,5)$. Aplicando Karnaugh con condiciones indiferentes:

Tabla de verdad:

a	b	c	$f(c, b, a)$
0	0	0	1
0	0	1	X
0	1	0	X
0	1	1	0
1	0	0	0
1	0	1	X
1	1	0	1
1	1	1	1

Circuito lógico:

The diagram shows a logic circuit with three inputs: a, b, and c. Inputs a and b are connected to NOT gates. The output of the NOT gate for a is connected to input a' of a 2-input AND gate. The output of the NOT gate for b is connected to input b of a 2-input AND gate. Input c is connected to input c of a 2-input AND gate. The outputs of the two AND gates are connected to a 2-input OR gate. The output of the OR gate is labeled $f = ab + a'c$.

Simplificación por Karnaugh:

Karnaugh de 4 variables					
	bc	00	01	11	10
a					
0		1	X	0	X
1		0	X	1	1

$f = ab + a'c$

Resumen: Álgebra de Boole

01

Postulados de Huntington

Cierre, identidad, conmutatividad, distributividad, complemento y existencia de al menos dos elementos.

02

Leyes y Teoremas

Idempotencia, absorción, asociatividad, involución, adyacencia, De Morgan y expansión de Shannon.

03

Formas Canónicas

Minitérminos (Σm_i) y maxitérminos (ΠM_i).
Conversión mediante De Morgan.

04

Puertas Lógicas

NOT, AND, OR, NAND, NOR, XOR. NAND y NOR son conjuntos funcionalmente completos.

05

Simplificación por Karnaugh

Agrupación de 1s en potencias de 2 adyacentes usando código Gray. Permite obtener la forma mínima.